CdP	Examen de Rattrapage	Durée : 2h Credit: 2
Année Scolaire : 2024-2025	Épreuve : Introduction à la Programmation en Langage Python	Classe : B1 (IT)

Instructions

- Lisez attentivement chaque question avant d'y répondre.
- Écrivez vos réponses de manière claire et structurée.
- Les réponses doivent inclure des explications et des exemples de code lorsque nécessaire.

Exercice 1 : Compréhension et correction de code (4 points)

On vous donne le code suivant :

```
def calculer_somme(liste):
    total = 0
    for i in range(liste):
        total += i
    return total

nombres = [1, 2, 3, 4, 5]
resultat = calculer_somme(nombres)
print("Somme:", resultat)
```

1. Ce programme comporte une erreur qui l'empêche de s'exécuter correctement. Trouvez l'erreur et expliquez-la. (1 pt)
2. Corrigez le programme et donnez le résultat attendu après exécution. (2 pts)
3. Expliquez le fonctionnement du programme corrigé. (1 pt)

Exercice 2 : Structures de contrôle et fonctions (5 points)

On souhaite écrire un programme permettant de vérifier si un nombre est pair ou impair en utilisant une fonction.

1. Écrivez une fonction `pair_ou_impair(nombre)` qui prend un nombre en paramètre et affiche s'il est pair ou impair. (2 pts)
2. Ajoutez un test pour vérifier que l'utilisateur entre bien un nombre entier avant de l'envoyer à la fonction. (1 pt)
3. Expliquez pourquoi l'utilisation de `try-except` est utile dans ce cas. (1 pt)
4. Donnez un exemple d'appel de votre fonction avec un nombre saisi par l'utilisateur. (1 pt)

Exercice 3 : Manipulation des structures de données (5 points)

On vous donne la liste suivante contenant des notes d'étudiants :

```
notes = [12, 15, 9, 18, 10, 7, 20, 14, 16, 8]
```

1. Écrivez une instruction Python permettant d'afficher uniquement les notes supérieures ou égales à 10. (1 pt)

Dr Dzo M. Serge Afenyiveh
d.afenyiveh@univkara.net

2. Écrivez une fonction `moyenne_notes(liste_notes)` qui retourne la moyenne des notes contenues dans une liste. (2 pts)
3. Ajoutez une gestion d'erreur pour éviter une division par zéro si la liste est vide. (1 pt)
4. Testez votre fonction avec la liste fournie et donnez le résultat obtenu. (1 pt)

Exercice 4 : Manipulation des fichiers en Python (3 points)

On souhaite enregistrer les noms et notes d'étudiants dans un fichier texte.

1. Écrivez un programme permettant de créer un fichier `notes.txt` et d'y enregistrer les données suivantes : (2 pts)

```
Alice : 15
Bob : 12
Clara : 18
David : 9
```

2. Ajoutez un bloc `try-except` pour gérer les erreurs liées à l'ouverture du fichier. (1 pt)

Exercice 5 : Bonnes pratiques et documentation (3 points)

1. Expliquez pourquoi la documentation (`docstrings`) est essentielle dans un programme Python. (1 pt)
2. Donnez un exemple de `docstring` pour la fonction suivante : (1 pt)

```
def additionner(a, b):
    return a + b
```

3. Pourquoi est-il recommandé d'utiliser des noms de variables explicites ? Donnez un exemple comparatif de mauvaise et bonne pratique en nommage de variables. (1 pt)

Exercice Bonus (Facultatif) : Challenge Python (3 points)

Dans cet exercice, vous devez écrire un programme Python permettant de générer un mot de passe sécurisé en fonction d'une longueur donnée par l'utilisateur.

1. Demandez à l'utilisateur d'entrer la longueur du mot de passe souhaitée (minimum 8 caractères). (1 pt)
2. Générez un mot de passe aléatoire contenant des lettres (majuscules et minuscules), des chiffres et des caractères spéciaux. (1 pt)
3. Assurez-vous que le mot de passe généré contient au moins une lettre majuscule, un chiffre et un caractère spécial. (1 pt)

Indication : Vous pouvez utiliser le module `random` et la bibliothèque `string` pour générer le mot de passe.

```
import random
import string

def generer_mot_de_passe(longueur):
    ...
```

Note : Cet exercice est facultatif mais permet d'obtenir jusqu'à **3 points supplémentaires** si la solution est correcte et optimisée.

"Le secret du succès est de faire les choses communes exceptionnellement bien." John D. Rockefeller

Dr Dzo M. Serge Afenyiveh
d.afenyiveh@univkara.net

CdP	Examen de Rattrapage	Durée : 2h Credit: 2
Année Scolaire : 2024-2025	Épreuve : Introduction à la Programmation en Langage Python	Classe : B1 (IT)

Exercice 1 : Compréhension et correction de code (4 points)

1. Erreur dans le programme

L'erreur dans le code est dans la ligne :

```
for i in range(liste):
```

Le paramètre `liste` n'est pas un entier, mais une liste. `range()` attend un entier en argument, ce qui provoque une erreur.

2. Code corrigé et résultat attendu

```
def calculer_somme(liste):
    total = 0
    for i in liste:  # Correction ici
        total += i
    return total

nombres = [1, 2, 3, 4, 5]
resultat = calculer_somme(nombres)
print("Somme:", resultat)
```

Résultat attendu : `Somme: 15`

3. Explication du code corrigé

La fonction `calculer_somme()` parcourt directement les éléments de la liste au lieu d'utiliser `range()`, ce qui permet d'additionner correctement les valeurs.

Exercice 2 : Structures de contrôle et fonctions (5 points)

1. Fonction vérifiant si un nombre est pair ou impair

```
def pair_ou_impair(nombre):
    if nombre % 2 == 0:
        print(f"{nombre} est un nombre pair.")
    else:
        print(f"{nombre} est un nombre impair.")
```

2. Vérification que l'utilisateur entre un entier

```
try:
    nombre = int(input("Entrez un nombre entier : "))
    pair_ou_impair(nombre)
except ValueError:
    print("Erreur : veuillez entrer un nombre entier valide.")
```

3. Utilité de `try-except`

Le bloc `try-except` permet d'éviter une erreur si l'utilisateur entre une valeur non entière. Sans cette protection, le programme planterait.

Dr Dzo M. Serge Afenyiveh
d.afenyiveh@univkara.net

4. Exemple d'appel de la fonction

```
pair_ou_impair(7)
```

Sortie : `7 est un nombre impair.`

Exercice 3 : Manipulation des structures de données (5 points)

1. Affichage des notes supérieures ou égales à 10

```
for note in notes:
    if note >= 10:
        print(note)
```

2. Fonction de calcul de la moyenne

```
def moyenne_notes(liste_notes):
    if len(liste_notes) == 0:
        return 0  # Gestion du cas liste vide
    return sum(liste_notes) / len(liste_notes)
```

3. Gestion de la division par zéro

Le test `if len(liste_notes) == 0` empêche une division par zéro en retournant 0 si la liste est vide.

4. Test de la fonction

```
notes = [12, 15, 9, 18, 10, 7, 20, 14, 16, 8]
print("Moyenne:", moyenne_notes(notes))
```

Résultat : `Moyenne:  13.9`

Exercice 4 : Manipulation des fichiers en Python (3 points)

1. Création et enregistrement des notes dans un fichier

```
with open("notes.txt", "w") as fichier:
    fichier.write("Alice : 15\n")
    fichier.write("Bob : 12\n")
    fichier.write("Clara : 18\n")
    fichier.write("David : 9\n")
```

2. Gestion des erreurs avec `try-except`

```
try:
    with open("notes.txt", "w") as fichier:
        fichier.write("Alice : 15\nBob : 12\nClara : 18\nDavid : 9\n")
except IOError:
    print("Erreur lors de l'  criture  du fichier.")
```


Dr Dzo M. Serge Afenyiveh
d.afenyiveh@univkara.net

Exercice 5 : Bonnes pratiques et documentation (3 points)

1. Importance des docstrings

Les `docstrings` facilitent la compréhension et la maintenance du code en fournissant une documentation intégrée.

2. Exemple de `docstring`

```
def additionner(a, b):
    """
    Additionne deux nombres et retourne le r sultat.

    :param a: Premier nombre
    :param b: Deuxi me nombre
    :return: Somme des deux nombres
    """
    return a + b
```

3. Bonnes pratiques de nommage

Mauvaise pratique :

```
a = 10
b = 20
c = a + b
```

Bonne pratique :

```
nombre1 = 10
nombre2 = 20
somme = nombre1 + nombre2
```

Exercice Bonus (Facultatif) : Challenge Python (3 points)

1. Demande de la longueur du mot de passe

```
longueur = int(input("Entrez la longueur du mot de passe (min 8) : "))
if longueur < 8:
    print("La longueur minimale est de 8 caract res.")
```

2. Génération d'un mot de passe sécurisé

```
import random
import string

def generer_mot_de_passe(longueur):
    caracteres = string.ascii_letters + string.digits + string.punctuation
    return ''.join(random.choice(caracteres) for _ in range(longueur))
```

3. Assurer la présence de caractères spécifiques

```
def mot_de_passe_secure(longueur):
    while True:
        mdp = generer_mot_de_passe(longueur)
        if (any(c.isupper() for c in mdp) and
            any(c.isdigit() for c in mdp) and
            any(c in string.punctuation for c in mdp)):
            return mdp
```


Dr Dzo M. Serge Afenyiveh
d.afenyiveh@univkara.net

4. Exemple de génération

```
print("Mot de passe g n r :", mot_de_passe_secure(12))
```


Dr Dzo M. Serge Afenyiveh
d.afenyiveh@univkara.net